Lab 2 Report:

Iris Classification with Regression

Name:

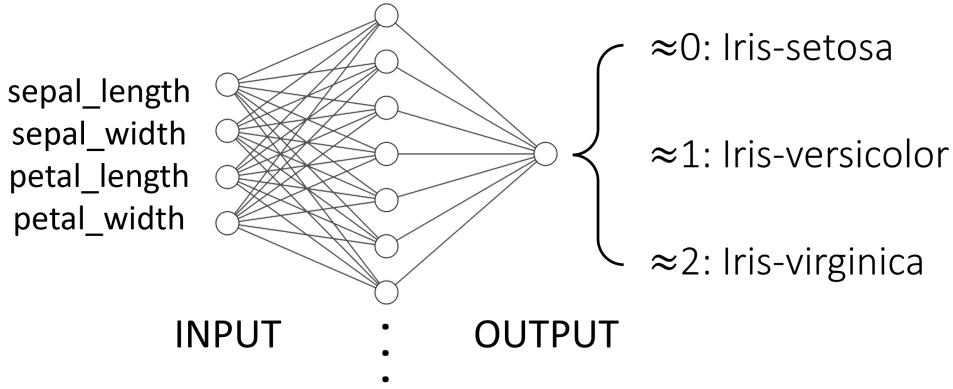
In [563…
```
# Import neccessary packages

%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt

import torch
```

In [564…
```
from IPython.display import Image # For displaying images in colab jupyter cell
```

In [565…
```
Image('lab2_exercise1.PNG', width = 1000)
```

Out [565…

Exercise 1: Iris Classification using Regression



In this exercise, you will train a neural network with a single hidden layer consisting of linear neurons to perform regression on iris datasets.

Your goal is to achieve a training accuracy of >90% under 50 epochs.

You are free to experiment with different data normalization methods, size of the hidden layer, learning rate and epochs.

You can round the output value to an integer (e.g. 0.34 -> 0, 1.78 -> 2) to compute the model accuracy.

Demonstrate the performance of your model via plotting the training loss and printing out the training accuracy. 42

Prepare Data

In [ ]:
```
from sklearn.datasets import load_iris

# iris dataset is available from scikit-learn package
iris = load_iris()

# Load the X (features) and y (targets) for training

X_train = iris['data']
y_train = iris['target']
```

```
# Load the name labels for features and targets

feature_names = iris['feature_names']
names = iris['target_names']

# Feel free to perform additional data processing here (e.g. standard scaling)
# Calculate the mean across columns

mean_x = np.mean(X_train, axis=0)

# Calculate the standard deviation across columns

std_x = np.std(X_train, axis=0)

# Standard scaling of x data by z = (x-mu)/std

X_train = (X_train - mean_x) / std_x
```

In [567…

```
# Print the first 10 training samples for both features and targets

print(X_train[:10, :], y_train[:10])
```

```
[[-0.90068117  1.01900435 -1.34022653 -1.3154443 ]
 [-1.14301691 -0.13197948 -1.34022653 -1.3154443 ]
 [-1.38535265  0.32841405 -1.39706395 -1.3154443 ]
 [-1.50652052  0.09821729 -1.2833891  -1.3154443 ]
 [-1.02184904  1.24920112 -1.34022653 -1.3154443 ]
 [-0.53717756  1.93979142 -1.16971425 -1.05217993]
 [-1.50652052  0.78880759 -1.34022653 -1.18381211]
 [-1.02184904  0.78880759 -1.2833891  -1.3154443 ]
 [-1.74885626 -0.36217625 -1.34022653 -1.3154443 ]
 [-1.14301691  0.09821729 -1.2833891  -1.44707648]] [0 0 0 0 0 0 0 0 0 0]
```

In [568…

```
# Print the dimensions of features and targets

print(X_train.shape, y_train.shape)
```

```
(150, 4) (150,)
```

In [569…

```
# feature_names contains name for each column in X_train
# For targets, 0 -> setosa, 1 -> versicolor, 2 -> virginica
```

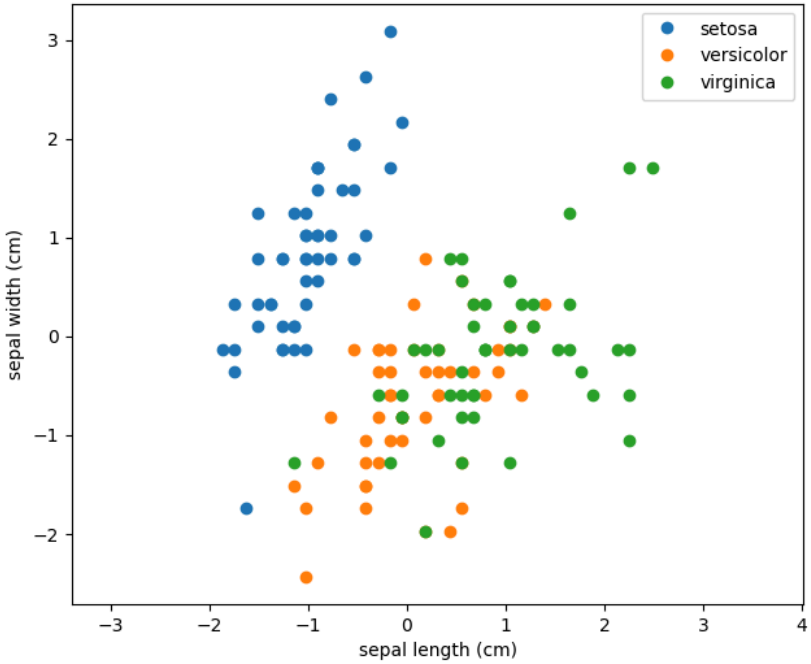
```
print(feature_names, names)
```

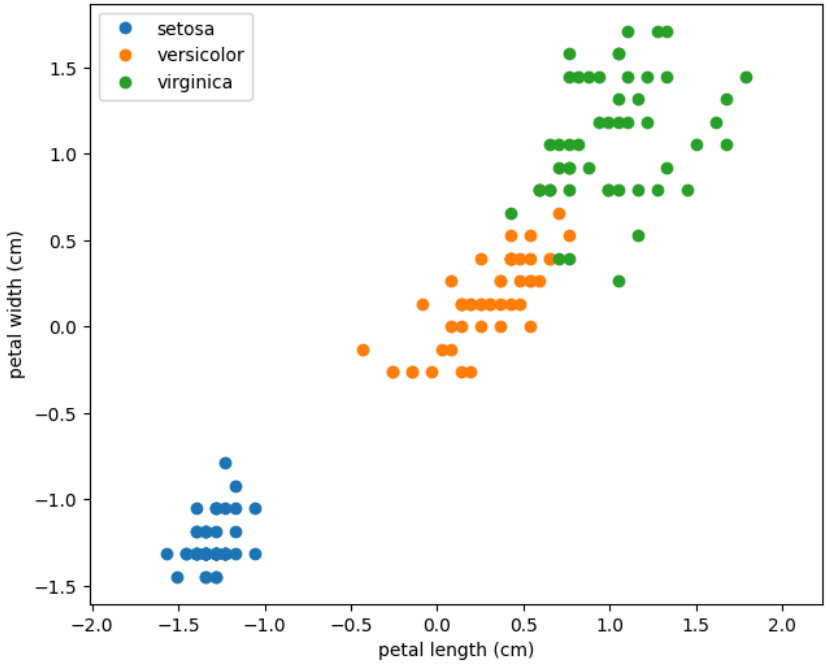
```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)'] ['setosa' 'versicolor'
'virginica']
```

In [ ]:

```
# We can visualize the dataset before training

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# enumerate picks up both the index (0, 1, 2) and the element ('setosa', 'versicolor', 'virginica') from "
# loop 1: target = 0, target_name = 'setosa'
# loop 2: target = 1, target_name = 'versicolor' etc

for target, target_name in enumerate(names):

    # Subset the rows of X_train that fall into each flower category using boolean mapping

    X_plot = X_train[y_train == target]

    # Plot the sepal length versus sepal width for the flower category

    ax1.plot(X_plot[:, 0], X_plot[:, 1], linestyle='none', marker='o', label=target_name)

# Label the plot
ax1.set_xlabel(feature_names[0])
ax1.set_ylabel(feature_names[1])
ax1.axis('equal')
ax1.legend()

# Repeat the above process but with petal length versus petal width
for target, target_name in enumerate(names):

    X_plot = X_train[y_train == target]

    ax2.plot(X_plot[:, 2], X_plot[:, 3], linestyle='none', marker='o', label=target_name)

ax2.set_xlabel(feature_names[2])
ax2.set_ylabel(feature_names[3])
ax2.axis('equal')
ax2.legend()
```

Out[ ]: <matplotlib.legend.Legend at 0x1697a2350>





Define Model

In [ ]:
```
class irisClassification(torch.nn.Module):

    def __init__(self, input_dim, output_dim):

        super(irisClassification, self).__init__()

        # First layer should input 4 features and give the dimension of the hidden layer

        self.layer1 = torch.nn.Linear(input_dim, 150) # Set hidden layer dimension to 150

        # Output dimension for each 4 features input should be 1, input dimension =output dimension of laye

        self.layer2 = torch.nn.Linear(150, output_dim)

    def forward(self, x):
```

```
        # YOUR CODE HERE
        # Final output is the output of layer2 from layer1 results input

        out = self.layer2(self.layer1(x))
        return out
```

Define Hyperparameters

In [572…

```
model = irisClassification(input_dim = 4, output_dim = 1)

learning_rate = 0.01
epochs  = 49 # This has to be <50

# We will use gradient descent for our optimizer and Mean Squared Error Loss function
loss_func = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr = learning_rate)
```

Identify Tracked Values

In [573…

```
# follow models performance over each epoch. Identify a metric and track it over epochs

train_loss_list = []
```

Train Model

In [574…

```
X_train = torch.from_numpy(X_train).float()
y_train = torch.from_numpy(y_train).float()

for epoch in range(epochs):

    optimizer.zero_grad()
    outputs = model(X_train) # train model using our X-training data

    # Unsqueeze y_train to broadcast output to the correct y training output

    loss = loss_func(outputs, y_train.unsqueeze(1))
    train_loss_list.append(loss.item())
```

```
    loss.backward()
    optimizer.step()
    print('epoch{}, loss{}'.format(epoch, loss.item()))
```

```
epoch0, loss1.4131736755371094
epoch1, loss0.42636698484420776
epoch2, loss0.2238367646932602
epoch3, loss0.14264975488185883
epoch4, loss0.10335128754377365
epoch5, loss0.08346481621265411
epoch6, loss0.07306299358606339
epoch7, loss0.06734884530305862
epoch8, loss0.06397466361522675
epoch9, loss0.06178880110383034
epoch10, loss0.06022493913769722
epoch11, loss0.05900324508547783
epoch12, loss0.05798409879207611
epoch13, loss0.05709658935666084
epoch14, loss0.056303512305021286
epoch15, loss0.05558439716696739
epoch16, loss0.05492705479264259
epoch17, loss0.05432353913784027
epoch18, loss0.05376811325550079
epoch19, loss0.053256288170814514
epoch20, loss0.05278429016470909
epoch21, loss0.05234883725643158
epoch22, loss0.051946982741355896
epoch23, loss0.05157605931162834
epoch24, loss0.05123364180326462
epoch25, loss0.05091748386621475
epoch26, loss0.05062554404139519
epoch27, loss0.05035591870546341
epoch28, loss0.050106871873140335
epoch29, loss0.04987679049372673
epoch30, loss0.049664206802845
epoch31, loss0.04946773871779442
epoch32, loss0.049286141991615295
epoch33, loss0.04911826178431511
epoch34, loss0.04896301403641701
epoch35, loss0.04881942272186279
epoch36, loss0.04868657886981964
epoch37, loss0.0485636405646801
epoch38, loss0.048449840396642685
epoch39, loss0.04834447056055069
epoch40, loss0.048246871680021286
epoch41, loss0.04815644398331642
```

```
epoch42, loss0.04807261750102043
epoch43, loss0.047994889318943024
epoch44, loss0.04792279005050659
epoch45, loss0.04785586893558502
epoch46, loss0.047793738543987274
epoch47, loss0.04773601144552231
epoch48, loss0.047682370990514755
```

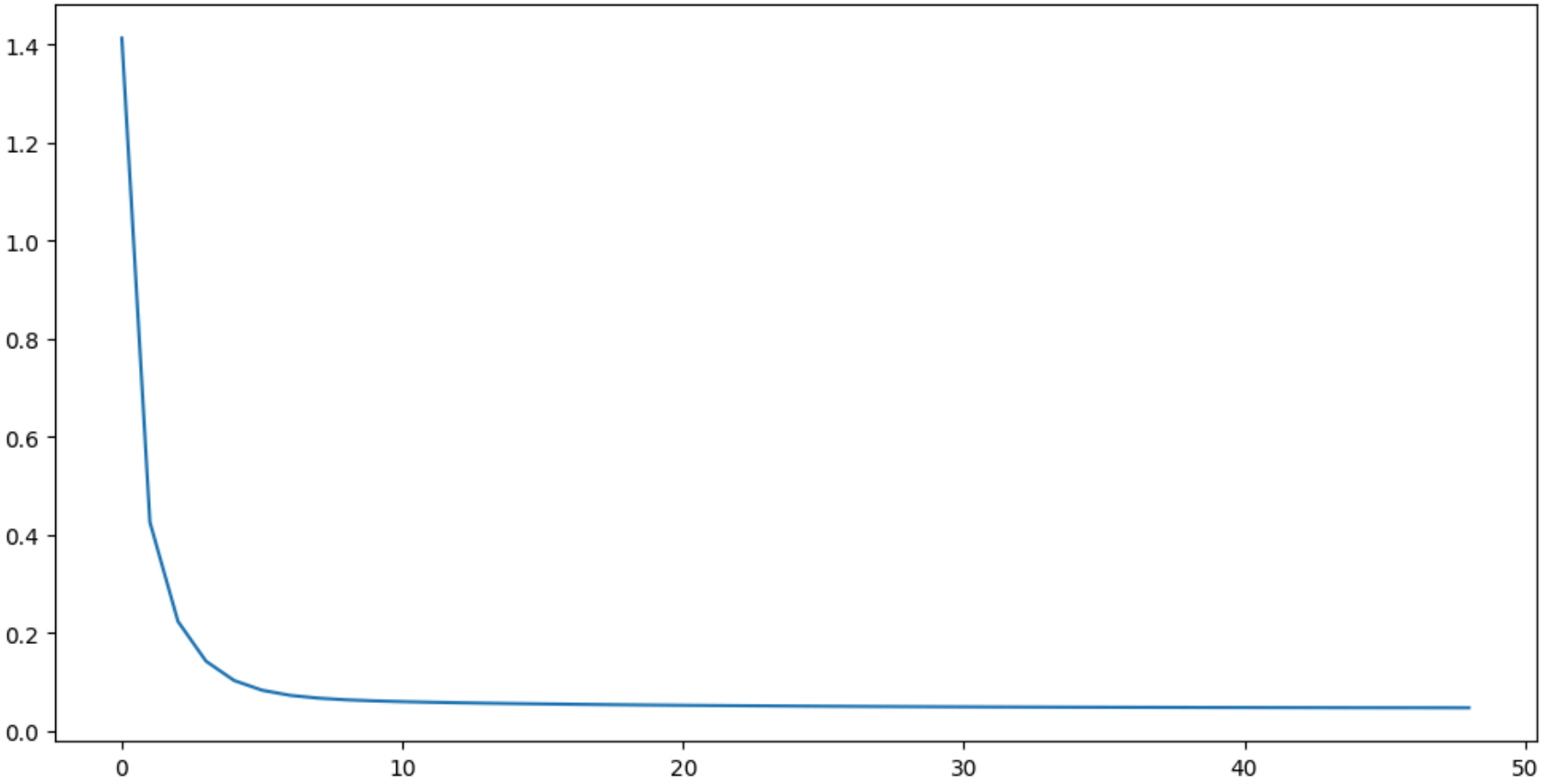
Visualize and Evaluate Model

In [575…

```
# Plot your training loss throughout the training
# Include proper x and y labels for the plot

plt.figure(figsize=(12, 6))

plt.plot(train_loss_list)
```

Out[575…

```
[<matplotlib.lines.Line2D at 0x16986b250>]
```



The training is not super accurate since both layers are linear and there's no activation function and dataset has only 150 points. However, the training loss exponentially decreased as training epochs increased.

In [ ]:
```
# Confirm that your model's training accuracy is >90%

with torch.no_grad():

    # Compare your model predictions with targets (y_train) to compute the training accuracy

    correct_predict = 0

    # You can round the model predictions to integer (e.g. 0.34 -> 0, 1.78 -> 2)

    predicted = model(X_train).numpy().round() # Round the predicted x value
```

```
    for i in range(len(predicted)):

        # Count for correct prediction if they match with each other
        if predicted[i] == y_train[i]:
            correct_predict += 1

    # Calculate training accuracy

    Training_accuracy = correct_predict / len(y_train)
    print(Training_accuracy)
```

```
0.98
```

In [ ]: